

7b - Programmazione CUDA – Factorize e Heat

I seguenti esercizi sulla programmazione CUDA possono essere svolti sul cluster HPC e/o sul PC se dotato di scheda NVIDIA.

I programmi si trovano su Elly e sul cluster in `cuda_factorize.tgz` e `cuda_heat.tgz`

cuda_factorize

Il sorgente `cuda_factorize.cu` determina uno dei due fattori del modulo utilizzando i `cuda-core` della GPU.

La libreria `BIGNUM` di `openssl`, utilizzata nelle altre implementazioni di `Factorize`, non è disponibile per `Cuda` per cui vengono utilizzati `unsigned long long integer`, con una limitazione per **il modulo a 64 bit**.

Se consideriamo l'elevato numero di test da calcolare non possiamo assegnare un thread per ogni test.

Se “M” è il modulo da fattorizzare, “tid” è l'identificativo del thread e “stride” è il numero di thread ($\text{stride} = \text{gridDim.x} * \text{blockDim.x}$) :

`for (unsigned long long i=tid*2+1; i<=max_check; i+=stride) <calcola M%i>`

Eseguire `cuda_factorize.slurm` sul PC o sul cluster.

Valutare la fattibilità ed eventualmente implementare altri algoritmi.

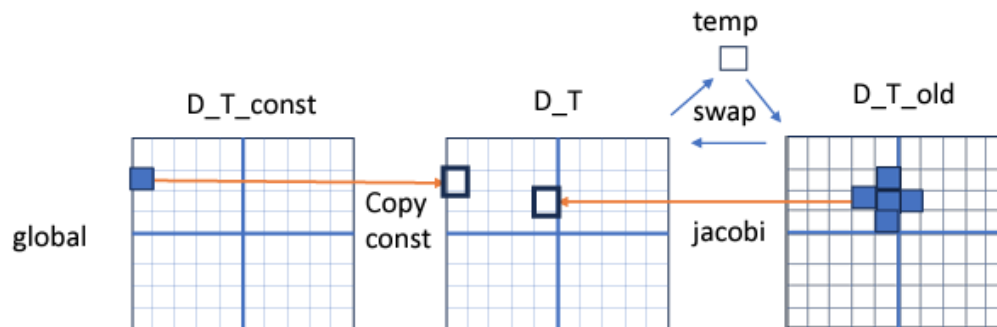
cuda_heat

Il programma `cuda_heat.cu` prima di iniziare le iterazioni crea 3 matrici nella memoria globale del Device GPU (application lifetime).

La matrice costante `D_T_const` contiene i valori di temperatura costante che devono essere ripristinati ad ogni iterazione.

La matrice `D_T_old` contiene i dati che verranno utilizzati per aggiornare `D_T` utilizzando `Jacobi`.

Verificare nel sorgente le chiamate a CUDA per l'allocazione delle matrici con `cudaMalloc()` e il caricamento dei dati con `cudaMemcpy()`



```
for(iter=0; iter<MAX_ITER; iter=iter+1)
{
    copy_constant<<<dimGrid, dimBlock>>>(d_T, d_T_const, NX, NY);
    cudaDeviceSynchronize();
    Jacobi_Iterator_GPU<<<dimGrid,dimBlock>>>(d_T, d_T_old, NX, NY);
    cudaDeviceSynchronize();

    temp=d_T;      d_T=d_T_old;      d_T_old=temp;    // swap
}
```

Il flusso del programma viene gestito dall'host che ad ogni iterazione esegue 2 kernel: il primo copia i dati costanti

`copy_constant<<<dimGrid,dimBlock>>>(..)` ,

il secondo calcola Jacobi : `Jacobi_iterator_GPU<<<dimGrid,dmBlock>>>(..)`

Verificare nel sorgente il funzionamento dei 2 kernel e le rispettive chiamate eseguite dall'host.

Consegna: Adattare ed eseguire `cuda_heat_scaling.slurm` e `cuda_heat_scaling.py` e generare `cuda_heat_scaling.png` per valutare la scalabilità del programma al variare della dimensione dei blocchi e della griglia.

Analizzare vantaggi, fattibilità ed eventualmente implementare nuove funzionalità per il miglioramento delle prestazioni:

- utilizzo di memoria shared e/o constant
- esecuzione di un singolo kernel per iterazione o per tutte le iterazioni

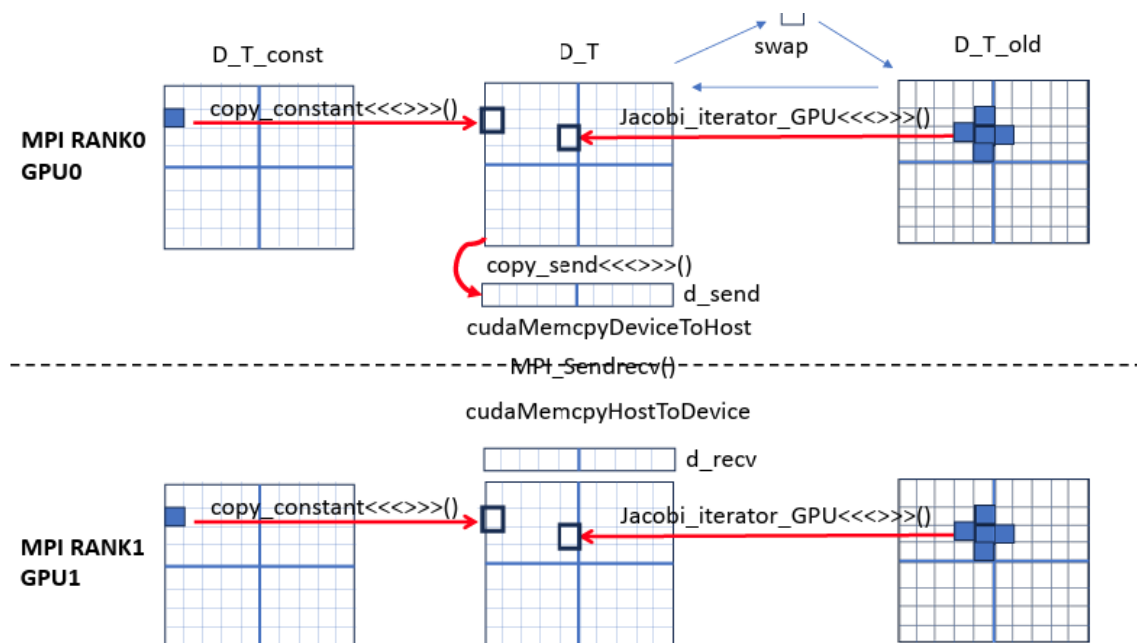
mpi+cuda heat

Il programma si trova in cuda/heat/MPI+cuda/

E' progettato per 2 GPU con comunicazione MPI per scambio dei dati di bordo.

Dimensione matrice NX 2NY

Trasferimento GPU0-> CPU (rank0) -> MPI -> CPU (rank1) -> GPU1



Per la compilazione:

module load gnu openmpi cuda

`nvcc main.cu -I/opt/ohpc/pub/mpi/openmpi-gnu/1.10.7/include \`

`-L/opt/ohpc/pub/mpi/openmpi-gnu/1.10.7/lib/ -lmpi`

Nella directory MPI+cuda/ si trovano 2 script slurm per l'esecuzione utilizzando 2 GPU sullo stesso nodo o 2 nodi con una GPU per nodo.

Ogni rank scrive un file di output che possiamo riunire con il seguente comando: `cat heatmap-0.dat heatmap-1.dat > heatmap.dat`

Per visualizzare il file di output: `python3 visualize`

Consegna:

Eseguire gli script

`mpi+cuda_heat_1x2.slurm` e `mpi+cuda_heat_2x1.slurm` ed esaminare i tempi del trasferimento delle righe di bordo nelle diverse fasi della comunicazione (`copy_send<<<>>>`, `cudaMemcpyDeviceToHost`, `MPI`, `cudaMemcpyHostToDevice`).